The Journal of Open Source Software

constrained_manipulability: A ROS 2 library to Compute and Visualize Constrained Capacities for Robotic Manipulators

Mark Zolotas [1*¶], **Philip Long** [2*], **Keerthi Sagar** [3], **and Taskin Padir** [1,4]

1 Northeastern University, USA (at the time of this work) **2** Atlantic Technological University, Ireland **3** Irish Manufacturing Research Limited, Mullingar, Ireland **4** Amazon Robotics, USA ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 15 August 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

This paper presents `constrained_manipulability`, a C++ library to compute and visualize a robot manipulator's constrained motion capacities during manipulation tasks. Manipulability polytopes are used to represent these capacities, providing a geometrical tool to evaluate the volume of free space surrounding the robotic arm when considering both environmental constraints and intrinsic robot constraints, such as collision with the environment and a manipulator's joint limits, respectively. Moreover, the polytopes defined by these convex constraints represent feasible configurations for the whole robot, that can be utilized in inverse kinematics (IK) optimization algorithms to produce collision-free motion trajectories.

The library is encapsulated in a Robot Operating System (ROS) package (Quigley et al., 2009), given that the robotics community widely relies on ROS as the standard for software development. We include ROS 1 and ROS 2 (Macenski et al., 2022) implementations of the core C++ library. The `constrained_manipulability` package also heavily depends on our `robot_collision_checking` package, a ROS interface for collision checking via the Flexible Collision Library (FCL) (Pan et al., 2012).

The main program of the `constrained_manipulability` package reads in a kinematic chain from a robot manipulator's Unified Robot Description Format (URDF), traversing from a user defined root of the robot (e.g., the robot's base frame) to a tip (e.g., the end-effector or tool). Joint position and velocity limits are also read into the program using this URDF input, while a collision world is maintained to express environmental constraints on the robot's motion. A variety of collision objects can be added or removed using common ROS message types, such as Octomaps (Hornung et al., 2013), mesh files or solid primitives. These joint limit and obstacle constraints are then stacked together to define different polytopes, such as the allowable motion polytope (Long et al., 2019) and the constrained velocity polytope (Long & Padır, 2018).

Statement of Need

Generating complex constraint geometries in an efficient and effective manner is a significant challenge for real-time robot manipulation. For common applications within manipulation, like motion planning or remote teleoperation, being able to represent the space of allowable motion around a robot's end-effector is a vital capability. Given such a representation of the obstacle-free volume around the robot's end-effector, a collision-free IK solution can be obtained to maximize the free space of a robot moving in a constrained environment. Manipulability polytopes represent manipulability capacities, e.g., for the robot to transmit velocities and

forces from the configuration space to the task space. The benefit of using convex polytopes is that they capture exact velocity bounds, rather than the approximation provided by other typical choices for performance indicators, such as ellipsoids (Yoshikawa, 1984). While the aforementionned ellipsoids have been the dominant paradigm due to historic computational constraints, more efficicent polytope generation methods (Sagar et al., 2023; Skuric et al., 2023) coupled with more computation availability has led to an increased usage.
Furthermore, since a polytope is defined by a set of inequality constraints, additional constraints can be easily incorporated into existing polytopes, e.g., mobile robot toppling constraints (Rasheed et al., 2018), friction cones (Caron et al., 2016), and maximum danger values.

The `constrained_manipulability` ROS 2 package aims to fulfil this need for constrained robot capacity calculation by supplying the robotics community with a fast C++ implementation that computes various types of polytopes. These geometrical constructs can also be visualized for workspace analysis or to guide an operator through the Cartesian motions available due to joint limits, kinematic constraints, and obstacles in the workspace, as illustrated in Figure 1. The utility of these visualizations has proven advantageous in remote teleoperation scenarios involving virtual reality headsets (Zolotas et al., 2021). Moreover, the package provides an interface to the `robot_collision_checking` package and a set of convenience functions for checking the distance/collision between a robot manipulator and the environment.

Figure 1: Collision-free path for a robot manipulator visualized as a trajectory of polytopes generated by maximizing the volume of allowable motion around the robot's end-effector.

Manipulability Polytopes Calculation

A polytope, $\mathcal{P}$, can be represented in two ways: as the convex hull of its vertex set, known as the $\mathcal{V}$-representation and denoted as $\mathcal{P}^V$, or as a volume bounded by a finite number of half-spaces, known as the $\mathcal{H}$-representation and denoted as $\mathcal{P}^H$. These are written respectively as:

$$\begin{aligned} \mathcal{P}^V &= \{\mathbf{x} : \mathbf{x} = \sum_{i=1}^{n} \alpha_i \mathbf{y}_i \Big| \alpha_i \geq 0, \sum_{i=1}^{n} \alpha_i = 1\}, \\ \mathcal{P}^H &= \mathbf{A}\mathbf{x} \leq \mathbf{b}, \end{aligned} \tag{1}$$

where $\mathbf{y}_i$ denotes the ith element of the vertex set, $\mathbf{x}$ is any point inside $\mathcal{P}$, $\mathbf{A}$ contains the half-spaces' normals, and $\mathbf{b}$ is the shifted distance from the origin along the normal. Converting between the $\mathcal{V}$ and $\mathcal{H}$ representations can be carried out in several ways, however the `constrained_manipulability` package uses the double description method (Fukuda & Prodon, 1996).

Below is a description of how constrained motion polytopes are constructed in `constrained_manipulability` for a given manipulator model.

Manipulator Model

Consider a serial manipulator with n degrees-of-freedom operating in 6-dimensional space. The end-effector pose is represented by a vector $\mathbf{x}_n \in \mathbb{R}^6$, consisting of the position $\mathbf{x}_n^p \in \mathbb{R}^3$ and unit quaternion orientation $\mathbf{x}_n^q \in \mathbb{R}^3$, all of which can be derived from the forward kinematic chain fk:

$$\mathbf{x}_n = fk(\mathbf{q}), \tag{2}$$

where $\mathbf{q} = [q_1, ..., q_n]$ are the joint configuration variables. The twist at the end-effector's n^{th} frame, ν_n, can then be obtained from the differential kinematic model:

$$\nu_n = \begin{bmatrix} \mathbf{v} \\ \omega \end{bmatrix} = \mathbf{J}_n \dot{\mathbf{q}}, \tag{3}$$

with $\mathbf{v}$ and ω denoting translational and angular velocities, respectively. $\mathbf{J}_n$ is the $6 \times n$ Jacobian matrix defined at the end-effector frame n and $\dot{\mathbf{q}} = [\dot{q}_1, ..., \dot{q}_n]^\top$ is the joint velocity vector.

Allowable Motions

At trajectory step k, the end-effector pose and the instantaneous joint velocity are given by $\mathbf{x}_n^k$ and $\dot{\mathbf{q}}^k$, respectively. If the end-effector travels a distance of $\delta\mathbf{x}_n^k$ over a period of δt seconds, then the resulting end-effector pose at instant $k+1$ can be obtained by linearizing Equation 2 using Equation 3:

$$\begin{aligned} \mathbf{x}_n^{k+1} &= \mathbf{x}_n^k + \delta\mathbf{x}_n^k \approx fk(\mathbf{q}^k + \delta\mathbf{q}^k), \\ \delta\mathbf{x}_n^k &= \mathbf{J}_n \delta\mathbf{q}^k, \end{aligned} \tag{4}$$

where $\delta\mathbf{q}^k$ denotes the displacement in joint variables over the timestep and is defined as:

$$\delta\mathbf{q}^k = \dot{\mathbf{q}}^k \delta t. \tag{5}$$

In a cluttered environment, the end-effector's allowable motion is conditioned on the allowable motion of its attached links. Even if the end-effector is located nominally in free space, certain allowable motions may be restricted by obstacles close to the chain's preceding links.

For a point $\mathcal{I}$ on the robot's kinematic chain whose position with respect to the world frame is denoted by the vector $\mathbf{r}_i$, the displacement due to $\delta\mathbf{q}^k$ is written as:

$$\delta\mathbf{x}_i^k = \mathbf{J}_i \delta\mathbf{q}^k, \tag{6}$$

DRAFT

where $\mathbf{J}_i \in \mathbb{R}^{3\times n}$ denotes the kinematic Jacobian matrix that relates velocities of antecedent joints along the kinematic chain to the translational velocity at point $\mathcal{I}$, with columns of zeros for joints that do not generate velocity at $\mathcal{I}$.

Using the linearization from Equation 6, limits of translational motion for any point on the manipulator body can be defined based on the location of environmental obstacles. Suppose there is an environmental obstacle $\mathcal{O}$ in the robot's proximity, with a location in the world frame described as $\mathbf{r}_o$. Then at instant k, the translational motion of point $\mathcal{I}$ towards $\mathbf{r}_o$, written as $\delta\mathbf{x}^k_{i,o}$, is defined as:

$$\delta\mathbf{x}^k_{i,o} = \hat{\mathbf{r}}^\top_{io}\mathbf{J}_i\delta\mathbf{q}^k, \tag{7}$$

where $\mathbf{r}_{io} = \mathbf{r}_i - \mathbf{r}_o$ is the relative position between $\mathcal{O}$ and $\mathcal{I}$, while $\hat{\mathbf{r}}_{io}$ is the corresponding normalized unit vector. Hence, to prevent a collision between $\mathcal{I}$ and $\mathcal{O}$, an allowable motion constraint defining the maximum displacement of $\mathcal{I}$ can be expressed as follows:

$$\hat{\mathbf{r}}^\top_{io}\mathbf{J}_i\delta\mathbf{q}^k \leq \|\mathbf{r}_{io}\|, \tag{8}$$

with $\|\mathbf{r}_{io}\|$ as the Euclidean norm of the vector $\mathbf{r}_{io}$. This inequality constraint can be repeated for a set of l points discretized along the kinematic chain, leading to the following set of linear inequalities:

$$\begin{bmatrix} \hat{\mathbf{r}}^T_{1o}\mathbf{J}_1 \\ \vdots \\ \hat{\mathbf{r}}^T_{lo}\mathbf{J}_l \end{bmatrix} \delta\mathbf{q}^k \leq \begin{bmatrix} \|\mathbf{r}_{1o}\| \\ \vdots \\ \|\mathbf{r}_{lo}\| \end{bmatrix}. \tag{9}$$

In our implementation, we select at every instant the point along each link nearest to an environmental obstacle, i.e., $l = n$. Hence, Equation 9 represents the set of instantaneous collision-free joint deviations for each link on the robot's kinematic chain, considering obstacle $\mathcal{O}$. We can extend this formulation to m obstacles in the robot's workspace, obtaining $(l \times m)$ constraints to ensure collision-free motions. This can be reduced by neglecting obstacles where $\|\mathbf{r}_{io}\|$ is greater than a threshold.

Aside from obstacles, a robot's motion is restricted by positional limits of the joints. To integrate these limits into our joint space polytope, the following linear inequalities are included:

$$\begin{bmatrix} \mathbb{I}_n \\ -\mathbb{I}_n \end{bmatrix} \delta\mathbf{q}^k \leq \begin{bmatrix} \mathbf{q}^{max} - \mathbf{q}^k \\ \mathbf{q}^k - \mathbf{q}^{min} \end{bmatrix}, \tag{10}$$

where $\mathbb{I}_n$ is the $n \times n$ identity matrix, while $\mathbf{q}^{max}$ and $\mathbf{q}^{min}$ are vectors of upper and lower positional joint limits, respectively.

As joint displacement increases, so does the linearization error. A maximum linearization limit is thus imposed:

$$\begin{bmatrix} \mathbb{I}_n \\ -\mathbb{I}_n \end{bmatrix} \delta\mathbf{q}^k \leq \begin{bmatrix} \delta\mathbf{q}^{max} \\ \delta\mathbf{q}^{max} \end{bmatrix}, \tag{11}$$

where $\delta\mathbf{q}^{max}$ is the vector of linearization limits for each joint of the kinematic chain. Increasing the values of $\delta\mathbf{q}^{max}$ expands the free space virtual fixture at a cost of reduced fidelity. While not necessary, for simplicity, we let $\delta q_1 = \delta q_2 = ... = \delta q_n = \epsilon_{lin}$ in our implementation, where ϵ_{lin} is a scalar linearization limit. Nevertheless, these linearization limits can also be altered at run time. For example, limits could be increased to enlarge the solution space of feasible configurations. Alternatively, the limits could be shrunk progressively in order to guide the user towards a defined goal configuration, as described in prior work on shared control with intention inference (Zolotas et al., 2021).

DRAFT

Constrained Motion Polytopes

The capacity of a serial manipulator can be obtained by first constructing the joint space polytope in the $\mathcal{H}$-representation according to its velocity limits:

$$\mathcal{Q}^H = \begin{bmatrix} \mathbb{I}_n \\ -\mathbb{I}_n \end{bmatrix} \dot{\mathbf{q}} \leq \begin{bmatrix} \dot{\mathbf{q}}^{max} \\ -\dot{\mathbf{q}}^{min} \end{bmatrix}, \tag{12}$$

where $\dot{\mathbf{q}}^{max}$ and $\dot{\mathbf{q}}^{min}$ are the robot's maximum and minimum joint velocities.

Using the double description method, an equivalent polytope in the $\mathcal{V}$-representation (i.e., defined by its vertices) is written as:

$$\mathcal{Q}^V = \{ \ \dot{\mathbf{q}}_1^v, \ \dot{\mathbf{q}}_2^v, \ \dots, \ \dot{\mathbf{q}}_q^v \ \}, \tag{13}$$

where $\dot{\mathbf{q}}_i^v$ denotes the ith vertex of the polytope $\mathcal{Q}$, given q vertices in n-dimensional space.

A linear transformation of a polytope produces another polytope. Ergo, a manipulability polytope, denoted as $\mathcal{MP}$, representing the Cartesian-space velocities can then be obtained by transforming the vertices of Equation 13 to Cartesian space using Equation 3. $\mathcal{MP}$'s vertex set representation of p vertices in 3-dimensional space is given as:

$$\mathcal{MP}^V = \{ \ \nu_1^v, \ \dots, \ \nu_p^v \ \} = \{ \ \mathbf{J}_n \dot{\mathbf{q}}_1^v, \ \dots, \ \mathbf{J}_n \dot{\mathbf{q}}_p^v \ \}, \tag{14}$$

with $\delta \mathbf{x}_j^v = \mathbf{J}_n \dot{\mathbf{q}}_j^v$ and $p \leq q$. The convexity of a polytope is preserved under affine transformation, thus a bounded volume of $\mathcal{MP}$ that represents the system's manipulability can easily be obtained to serve as an exact indicator of robot performance.

The above considers a constrained velocity polytope based on the constraints specified in Equation 12. To obtain the constrained motion polytope for a manipulator, a joint space polytope is first defined using position deviations instead of instantaneous velocities, following from its position limits stated in Equation 10.

Therefore, by stacking Equation 9, Equation 10, and Equation 11, an $\mathcal{H}$-representation of a joint space polytope that approximates, at any instant k, the maximum range of joint displacements with respect to the system's constraints is derived:

$$\mathbf{A}^k \delta \mathbf{q}^k \leq \mathbf{b}^k. \tag{15}$$

The polytope can then be transformed to a $\mathcal{V}$-representation using the double description method (Fukuda & Prodon, 1996), as in Equation 13, after which the Cartesian task space representation is obtained using the differential kinematic model Equation 3, as in Equation 14. Overall, Equation 15 defines a set of Cartesian displacements for the manipulator's end effector, at an instant k, for which an IK solution can be obtained that is collision-free and within the positional joint limits.

Block Diagram

Any joint position and velocity limits are extracted from the robot's URDF description via ROS 2 parameter operations. Polytopes are calculated as described above using these limits and by obtaining the minimum distance from each link on the robot to objects in the collision world. FCL (Pan et al., 2012) is required to compute these distances and is accessible via the interface package robot_collision_checking. Polytopes in Cartesian space can then be returned from getter functions, like:

```
Polytope getConstrainedAllowableMotionPolytope(
  const sensor_msgs::msg::JointState& joint_state,
  bool show_polytope,
  Eigen::MatrixXd& AHrep,
```

DRAFT

```
  Eigen::VectorXd& bHrep,
  Eigen::Vector3d& offset_position,
  const std::vector<double>& color_pts,
  const std::vector<double>& color_line);
```

where `AHrep` and `bHrep` represent the joint space polytope constraints defined in Equation 15.

A diagram summarizing the `constrained_manipulability` package architecture and ROS 2 communication flow is displayed in Figure 2.

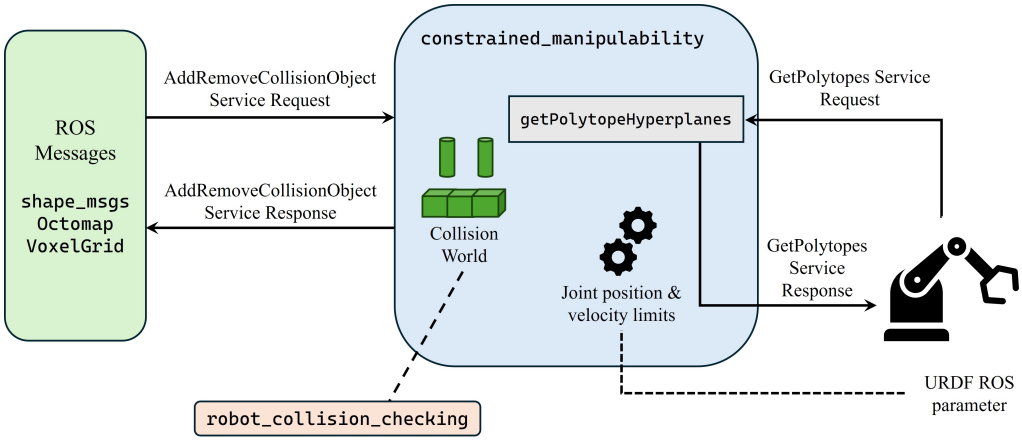

Figure 2: Block diagram for the constrained_manipulability ROS 2 package.

Acknowledgements

Mark Zolotas is currently at Toyota Research Institute (TRI), Cambridge, MA, USA. This paper describes work performed at Northeastern University and is not associated with TRI.

Taskin Padir holds concurrent appointments as a Professor of Electrical and Computer Engineering at Northeastern University and as an Amazon Scholar. This paper describes work performed at Northeastern University and is not associated with Amazon.

References

Caron, S., Pham, Q.-C., & Nakamura, Y. (2016). Zmp support areas for multicontact mobility under frictional constraints. *IEEE Transactions on Robotics*, *33*(1), 67–80.

Fukuda, K., & Prodon, A. (1996). Double description method revisited. *Combinatorics and Computer Science*, *1120*, 91.

Hornung, A., Wurm, K. M., Bennewitz, M., Stachniss, C., & Burgard, W. (2013). OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous Robots*, *34*, 189–206.

Long, P., Keleştemur, T., Önol, A. Ö., & Padir, T. (2019). Optimization-based human-in-the-loop manipulation using joint space polytopes. *International Conference on Robotics and Automation*, 204–210.

Long, P., & Padır, T. (2018). Evaluating robot manipulability in constrained environments by velocity polytope reduction. *IEEE-RAS International Conference on Humanoid Robots*, 497–502.

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot operating

DRAFT

system 2: Design, architecture, and uses in the wild. *Science Robotics*, *7*(66), eabm6074. https://doi.org/10.1126/scirobotics.abm6074

Pan, J., Chitta, S., & Manocha, D. (2012). FCL: A general purpose library for collision and proximity queries. *IEEE International Conference on Robotics and Automation*, 3859–3866. https://doi.org/10.1109/ICRA.2012.6225337

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. (2009). ROS: An open-source robot operating system. *ICRA Workshop on Open Source Software*, *3*.

Rasheed, T., Long, P., Marquez-Gamez, D., & Caro, S. (2018). Tension distribution algorithm for planar mobile cable-driven parallel robots. *Cable-Driven Parallel Robots: Proceedings of the Third International Conference on Cable-Driven Parallel Robots*, 268–279.

Sagar, K., Caro, S., Padır, T., & Long, P. (2023). Polytope-based continuous scalar performance measure with analytical gradient for effective robot manipulation. *IEEE Robotics and Automation Letters*, *8*(11), 7289–7296. https://doi.org/10.1109/LRA.2023.3313926

Skuric, A., Padois, V., & Daney, D. (2023). Pycapacity: A real-time task-space capacity calculation package for robotics and biomechanics. *Journal of Open Source Software*, *8*(89), 5670. https://doi.org/10.21105/joss.05670

Yoshikawa, T. (1984). Analysis and control of robot manipulators with redundancy. *International Symposium on Robotics Research*, 735–747.

Zolotas, M., Wonsick, M., Long, P., & Padır, T. (2021). Motion Polytopes in Virtual Reality for Shared Control in Remote Manipulation Applications. *Frontiers in Robotics and AI*, *8*, 286. https://doi.org/10.3389/frobt.2021.730433

DRAFT